

Week 2 at a glance

Textbook reading: Sections 1.1, 1.2

Before Monday, read Figure 1.4 and Definition 1.5 (definition of finite automata) on pages 34-35.

Before Wednesday, read pages 41-43 (Figures 1.18, 1.19, 1.20) for examples of automata and languages. Wednesday's class will be asynchronous. Please read over the annotated notes and watch the relevant supplementary videos. Ask any followup questions in the discussion forum or in office hours.

Before Friday, read pages 48-50 (Figures 1.27, 1.29) which introduces nondeterminism.

No class on Week 3 Monday in observance of Martin Luther King Jr. Day.

For Week 3 Wednesday: read the definition of the union, concatenation, and star operations for languages, given as Definition 1.23 on page 44 and a useful example is Example 1.24.

We will be learning and practicing to:

- Clearly and unambiguously communicate computational ideas using appropriate formalism. Translate across levels of abstraction.
 - Give examples of sets that are regular (and prove that they are).
 - * **State the definition of the class of regular languages**
 - * **Give examples of regular languages, using each of the three equivalent models of computation for proving regularity.**
 - Describe and use models of computation that don't involve state machines.
 - * **Given a DFA, find a regular expression that describes its language.**
 - * **Given a regular expression, find a DFA that recognizes its language.**
 - Use precise notation to formally define the state diagram of finite automata.
 - Use clear English to describe computations of finite automata informally.
 - * **State the formal definition of (deterministic) finite automata**
 - * **Trace the computation of a finite automaton on a given string using its state diagram**
 - * **Translate between a state diagram and a formal definition**
 - * **Determine if a given string is in the language recognized by a finite automaton**
 - * **Design an automaton that recognizes a given language**
 - * **Specify a general construction for DFA based on parameters**
 - * **Design general constructions for DFA**

TODO:

#FinAid Assignment on Canvas (complete as soon as possible) and read syllabus on Canvas

Schedule your Test 1 Attempt 1, Test 2 Attempt 1, Test 1 Attempt 2, and Test 2 Attempt 2 times at PrairieTest (<http://us.prairietest.com>)

Review Quiz 1 on PrairieLearn (<http://us.prairielearn.com>), complete by 1/15/25

Create a homework group, possibly by using the Piazza (<https://piazza.com/>) find-a-teammate tool

Homework 1 submitted via Gradescope (<https://www.gradescope.com/>), due 1/16/25

Review Quiz 2 on PrairieLearn (<http://us.prairielearn.com>), complete by 1/22/25

Week 2 Monday: Finite automata

****This definition was in the pre-class reading**** A finite automaton (FA) is specified by $M = (Q, \Sigma, \delta, q_0, F)$. This 5-tuple is called the **formal definition** of the FA. The FA can also be represented by its state diagram: with nodes for the state, labelled edges specifying the transition function, and decorations on nodes denoting the start and accept states.

Finite set of states Q can be labelled by any collection of distinct names. Often we use default state labels $q_0, q_1, \dots$

The alphabet Σ determines the possible inputs to the automaton. Each input to the automaton is a string over Σ , and the automaton “processes” the input one symbol (or character) at a time.

The transition function δ gives the next state of the automaton based on the current state of the machine and on the next input symbol.

The start state q_0 is an element of Q . Each computation of the machine starts at the start state.

The accept (final) states F form a subset of the states of the automaton, $F \subseteq Q$. These states are used to flag if the machine accepts or rejects an input string.

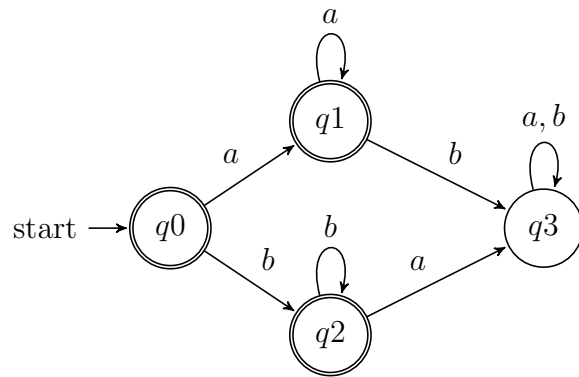
The computation of a machine on an input string is a sequence of states in the machine, starting with the start state, determined by transitions of the machine as it reads successive input symbols.

The finite automaton M accepts the given input string exactly when the computation of M on the input string ends in an accept state. M rejects the given input string exactly when the computation of M on the input string ends in a nonaccept state, that is, a state that is not in F .

The language of M , $L(M)$, is defined as the set of all strings that are each accepted by the machine M . Each string that is rejected by M is not in $L(M)$. The language of M is also called the language recognized by M .

What is **finite** about all finite automata? (Select all that apply)

- ☐ The size of the machine (number of states, number of arrows)
- ☐ The length of each computation of the machine
- ☐ The number of strings that are accepted by the machine

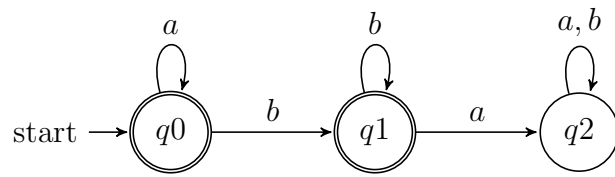


The formal definition of this FA is

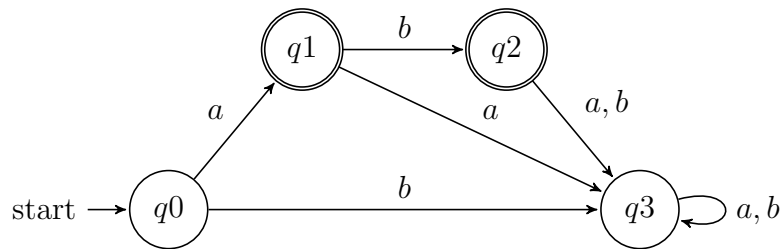
Classify each string $a, aa, ab, ba, bb, \varepsilon$ as accepted by the FA or rejected by the FA.

Why are these the only two options?

The language recognized by this automaton is



The language recognized by this automaton is



The language recognized by this automaton is

Week 2 Wednesday: Finite automaton constructions - asynchronous

Review: Formal definition of DFA: $M = (Q, \Sigma, \delta, q_0, F)$

- Finite set of states Q
- Alphabet Σ
- Transition function δ
- Start state q_0
- Accept (final) states F

Quick check: In the state diagram of M , how many outgoing arrows are there from each state?

Note: We'll see a new kind of finite automaton. It will be helpful to distinguish it from the machines we've been talking about so we'll use **Deterministic Finite Automaton** (DFA) to refer to the machines from Section 1.1.

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_0\})$ where δ is (rows labelled by states and columns labelled by symbols):

δ	a	b
q_0	q_1	q_1
q_1	q_2	q_2
q_2	q_0	q_0

The state diagram for M is

Give two examples of strings that are accepted by M and two examples of strings that are rejected by M :

A regular expression describing $L(M)$ is

A state diagram for a finite automaton recognizing

$$\{w \mid w \text{ is a string over } \{a, b\} \text{ whose length is not a multiple of } 3\}$$

Extra example: Let n be an arbitrary positive integer. What is a formal definition for a finite automaton recognizing

$$\{w \mid w \text{ is a string over } \{0, 1\} \text{ whose length is not a multiple of } n\}?$$

Consider the alphabet $\Sigma_1 = \{0, 1\}$.

A state diagram for a finite automaton that recognizes $\{w \mid w \text{ contains at most two 1's}\}$ is

A state diagram for a finite automaton that recognizes $\{w \mid w \text{ contains more than two 1's}\}$ is

Strategy: Add “labels” for states in the state diagram, e.g. “have not seen any of desired pattern yet” or “sink state”. Then, we can use the analysis of the roles of the states in the state diagram to work towards a description of the language recognized by the finite automaton.

Or: decompose the language to a simpler one that we already know how to recognize with a DFA or NFA.

Textbook Exercise 1.14: Suppose A is a language over an alphabet Σ . If there is a DFA M such that $L(M) = A$ then there is another DFA, let's call it M' , such that $L(M') = \overline{A}$, the complement of A , defined as $\{w \in \Sigma^* \mid w \notin A\}$.

Proof idea:

A useful bit of terminology: the **iterated transition function** of a finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ is defined recursively by

$$\delta^*(q, w) = \begin{cases} q & \text{if } q \in Q, w = \varepsilon \\ \delta(q, a) & \text{if } q \in Q, w = a \in \Sigma \\ \delta(\delta^*(q, u), a) & \text{if } q \in Q, w = ua \text{ where } u \in \Sigma^* \text{ and } a \in \Sigma \end{cases}$$

Using this terminology, M accepts a string w over Σ if and only if $\delta^*(q_0, w) \in F$.

Proof:

Week 2 Friday: Nondeterministic automata

We saw that whenever a language is recognized by a DFA, its complement is also recognized by some (other) DFA.

Another way to say this is that the collection of languages that are each recognizable by a DFA is **closed** under complementation.

Before we continue with more complicated constructions, let's revisit some of the assumption of the **DFA** model.

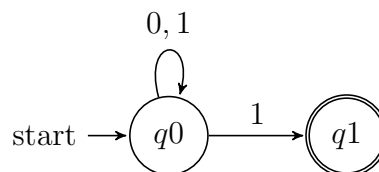
Recall that the computation of **deterministic** finite automaton has exactly one choice for its next step given the current state and the current character to read.

Nondeterministic finite automaton (Sipser Page 53) Given as $M = (Q, \Sigma, \delta, q_0, F)$

Finite set of states Q	Can be labelled by any collection of distinct names. Default: $q_0, q_1, \dots$
Alphabet Σ	Each input to the automaton is a string over Σ .
Arrow labels Σ_ϵ	$\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$. Arrows in the state diagram are labelled either by symbols from Σ or by ϵ
Transition function δ	$\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ gives the set of possible next states for a transition from the current state upon reading a symbol or spontaneously moving.
Start state q_0	Element of Q . Each computation of the machine starts at the start state.
Accept (final) states F	$F \subseteq Q$.

M accepts the input string $w \in \Sigma^*$ if and only if **there is** a computation of M on w that processes the whole string and ends in an accept state.

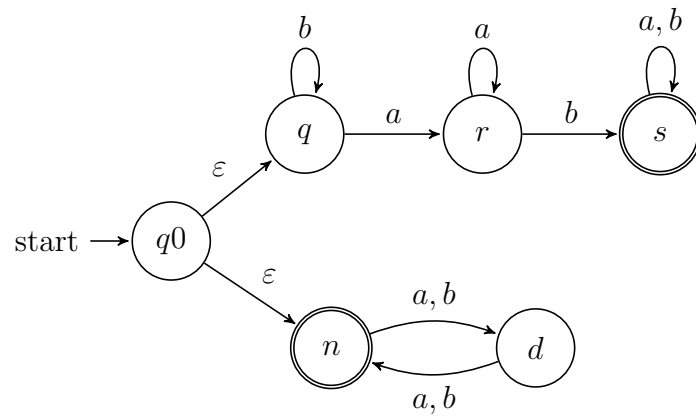
The formal definition of the NFA over $\{0, 1\}$ given by this state diagram is:



The language over $\{0, 1\}$ recognized by this NFA is:

Practice: Change the transition function to get a different NFA which accepts the empty string (and potentially other strings too).

The state diagram of an NFA over $\{a, b\}$ is:



The formal definition of this NFA is:

Suppose A_1, A_2 are languages over an alphabet Σ . **Claim:** if there is a NFA N_1 such that $L(N_1) = A_1$ and NFA N_2 such that $L(N_2) = A_2$, then there is another NFA, let's call it N , such that $L(N) = A_1 \cup A_2$.

Proof idea: Use nondeterminism to choose which of N_1, N_2 to run.

Formal construction: Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ and assume $Q_1 \cap Q_2 = \emptyset$ and that $q_0 \notin Q_1 \cup Q_2$. Construct $N = (Q, \Sigma, \delta, q_0, F_1 \cup F_2)$ where

- $Q =$
- $\delta : Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q)$ is defined by, for $q \in Q$ and $x \in \Sigma_\varepsilon$:

Proof of correctness would prove that $L(N) = A_1 \cup A_2$ by considering an arbitrary string accepted by N , tracing an accepting computation of N on it, and using that trace to prove the string is in at least one of A_1, A_2 ; then, taking an arbitrary string in $A_1 \cup A_2$ and proving that it is accepted by N . Details left for extra practice.